Files

Variables used in a program have a lifetime that lasts till the time the program is under execution. Thus we need a solution for permanent storage.
Data are stored permanently on secondary storage devices for re-usability. The data entered, and the output can be stored permanently into a file.
A **file** is a named location on a secondary storage media where data are permanently stored for later access.

Types of files

Computers store every file as a collection of 0s and 1s i.e., in binary form. Therefore, every file is basically just a series of bytes stored one after the other.
There are mainly two types of data files — **text file** and **binary file**.
A text file consists of human readable characters, which can be opened by any text editor.
On the other hand, binary files are made up of non-human readable characters and symbols, which require specific programs to access its contents.

Text file

A text file can be understood as a sequence of characters consisting of alphabets, numbers and other special symbols.
Files with extensions like `.txt`, `.py`, `.csv`, etc. are some examples of text files.

In ASCII, UNICODE or any other encoding scheme, the value of each character of the text file is stored as bytes.
While opening a text file, the text editor translates each ASCII value and shows us the equivalent character that is readable by the human being. For example, the ASCII value 65 binary equivalent 1000001) will be displayed by a text editor as the letter 'A' since the number 65 in ASCII character set represents 'A'.

Each line of a text file is terminated by a special character, called the **End of Line (EOL)**. For example, the default **EOL** character in Python is

the newline (\n).
When a text editor or a program interpreter encounters the ASCII equivalent of the EOL character, it displays the remaining file contents starting from a new line.
Contents in a text file are usually separated by white space, but comma (,) and tab (\t) are also commonly used to separate values in a text file.

Binary file

Binary files are also stored in terms of bytes (0s and 1s) but unlike text files, these bytes do not represent some ASCII value of a character. Rather, they represent the actual content such as image, audio, video, compressed version of other files, executable files etc. These files are not human readable.

Trying to open a binary file using a text editor will show some garbage values. We need to use specific software that is able to read or write the contents of that particular binary file.

Even a single bit change can corrupt the file and make it unreadable to the supporting application. Also, it is difficult to remove any error which may occur in the binary file as the stored contents are not human readable.

Opening and Closing a Text File

We broadly access files either to write or read data from it. But operations on files include creating and opening a file, writing data in a file, traversing a file, reading data from a file and so on.
Python has the `io` module that contains different functions for handling files.

Opening a file

To open a file in Python, we use the `open()` function.
The syntax of `open()` is as follows:

```
file_object = open(file_name, access_mode)
```

- This function returns a file object called **file handle**.

- We can use this file object to transfer data to and from the file (read and write) by calling the functions defined in the Python's `io` module.
- The `file_name` should be the name of the file that has to be opened. If the file is not in the current working directory, then we need to specify the complete path of the file along with its name.
- The `access_mode` is an optional argument that represents the mode in which the file has to be accessed by the program. It is also referred to as **processing mode**. Here mode means the operation for which the file has to be opened. The default access mode is the **read mode**.
- By default, files are opened in **text mode** that means strings can be read or written. Files containing non-textual data are opened in **binary mode** that means read/write are performed in terms of bytes.

File Mode	Description	File Offset position
<r>	Opens the file in read-only mode.	Beginning of the file
<rb>	Opens the file in binary and read-only mode.	Beginning of the file
<r+> or <+r>	Opens the file in both read and write mode.	Beginning of the file
<w>	Opens the file in write mode. If the file already exists, all the contents will be overwritten. If the file doesn't exist, then a new file will be created.	Beginning of the file
<wb+> or <+wb>	Opens the file in read,write and binary mode. If the file already exists, the contents will be overwritten. If the file doesn't exist, then a new file will be created.	Beginning of the file
<a>	Opens the file in append mode. If the file doesn't exist, then a new file will be created.	End of the file
<a+> or <+a>	Opens the file in append and read mode. If the file doesn't exist, then it will create a new file.	End of the file

  - The file offset position in the table refers to the position of the file object when the file is opened in a particular mode.

> `r+` : read and start overwriting at the start of the file. The number of characters overwritten depends on the number of characters being written.
> `a+` : read and append at the end of the file
> `w+` : read and write. The original content is lost and the new data is overridden.

File Attributes

1. `closed` -> returns `True` if file is closed & `False` otherwise

2. `mode` -> returns the access mode in which the file was opened
3. `name` -> returns name of the file

Closing a file

Once we are done with the read/write operations on a file, it is a good practice to close the file. Python provides a `close()` method to do so. While closing a file, the system frees the memory allocated to it.
The syntax of `close()` is:

```
file_object.close()
```

Python makes sure that any unwritten or unsaved data is flushed off (written) to the file before it is closed. Hence, it is always advised to close the file once our work is done.
If the file object is re-assigned to some other file, the previous file is automatically closed.

Opening a file using `with` clause

The syntax of with clause is:

```
with open (file_name, access_mode) as file_object:
```

The advantage of using with clause is that any file that is opened using this clause is closed automatically, once the control comes outside the with clause.
In case the user forgets to close the file explicitly or if an exception occurs, the file is closed automatically.
Also, it provides a simpler syntax

```
with open('my-text-file.txt') as f:
    content = f.read()
```

Reading from a text file

Before reading a file, we must make sure that the file is opened in “r”, “r+”, “w+” or “a+” mode. There are three ways to read the contents of a file:

The `read(n)` method

This method is used to read a specified number of bytes of data from a data file.
The syntax of read() method is:

```
file_object.read(n)
```

- where `n` is the number of bits to be read
- if `n` is not specified or negative number is used then the entire file is read

The `readline(n)` method

This method reads one complete line from a file where each line terminates with a newline character.
It can also be used to read a specified number (n) of bytes of data from a file but maximum up to the newline character.
If no argument or a negative number is specified, it reads a complete line and returns string.

```
>>> myobject=open("myfile.txt",'r')
>>> myobject.readline(10)
'Hello ever'
>>> myobject.close()
```

To read the entire file line by line using the `readline()`, we can use a loop. This process is known as looping/iterating over a file object. It returns an empty string when `EOF` is reached.

```
f = open('groceries.txt')
line = f.readline()

while line:
    print(line)
    line = f.readline()

f.close()
```

The `readlines()` method

The method reads all the lines and returns the lines along with newline as a list of strings.

```
f = open('groceries.txt', encoding='utf-8')

lines = f.readlines()
for line in lines:
    print(line)

f.close()
```

Setting offsets in a file

If we want to access data in a random fashion, then Python gives us `seek()` and `tell()` functions to do so.

The `tell()` method

This function returns an integer that specifies the current position of the file object in the file. The position so specified is the byte position from the beginning of the file till the current position of the file object.

```
# Syntax
file_object.tell()
```

The `seek()` method

This method is used to position the file object at a particular position in a file.

```
# Syntax
file_object.seek(offset, [, reference_point])
```

- `offset` -> the number of bytes by which the file object is to be moved.
- `reference_point` -> the starting position of the file object. That is, with reference to which position, the offset has to be counted. It can have any of the following values:
  - `0` -> beginning of the file
  - `1` -> current position of the file

    - 2 -> end of file
- By default, the value of `reference_point` is `0`

Writing to a text file

For writing to a file, we first need to open it in write or append mode. If we open an existing file in write mode, the previous data will be erased, and the file object will be positioned at the beginning of the file.
On the other hand, in append mode, new data will be added at the end of the previous data as the file object is at the end of the file.
After opening the file, we can use the following methods to write data in the file.

The `write()` method

`write()` method takes a string as an argument and writes it to the text file.
It returns the number of characters being written.
Also, we need to add a newline character (`\n`) at the end of every sentence to mark the end of line.

> `\n` is treated as a single character

```
filename = 'my-text-file.txt'
content = 'Hello from the other side'

with open(filename, 'w') as f:
    f.write(content + '\n')
```

If numeric data are to be written to a text file, the data need to be converted into string before writing to the file. For example:

```
filename = 'my-text-file.txt'

with open(filename, 'w') as f:
    f.write('What is the current year?\n')
    f.write(str(2026) + '\n')
```

The `write()` actually writes data onto a buffer. When the `close()` method is executed, the contents from this buffer are moved to the file located on the permanent storage.

The `writelines()` method

This method is used to write multiple strings to a file.
We need to pass an iterable object like lists, tuple, etc. containing strings to the `writelines()` method.
The `writelines()` method does not return the number of characters written in the file.

```
filename = 'my-text-file.txt'
lines = ['What is the current year?\n', 'I answered,
"2026"\n']

with open(filename, 'w') as f:
    f.writelines(lines)
```

Using append(`a`) mode

```
with open('my-text-file.txt', 'a') as f:
    f.write('How are you finding your new job?\n')
    f.write('Eh...it\'s a job, just like every...')
```

- The above program will add the two lines at the end of the file
- If the file `my-text-file.txt` doesn't exist, it will be created just as with write(`w`) mode.

File handling

Renaming and Deleting files

- In Python, you can rename and delete files using built-in functions from the `os` module.

Renaming files

- To rename a file in Python, you can use the `os.rename()` function
- This function takes two arguments: path of the file to be renamed and the new filename.

- Syntax: `os.rename(path, new_filename)`
- usage example

```
from os import rename

new_filename = input('enter new filename: ')
rename('./test.txt', new_filename)
```

Deleting files in python

- We can delete a file in python using `os.remove()` function.
- This function deletes a file specified by its path.
- Syntax: `os.remove(path)`

```
from os import remove

remove('./something.txt')
```

The `seek()` method

This method is used to position the file object at a particular position in a file.

```
# Syntax
file_object.seek(offset[, reference_point])
```

- `offset` -> the number of bytes by which the file object is to be moved.
- `reference_point` -> the starting position of the file object. That is, with reference to which position, the offset has to be counted. It can have any of the following values:
  - 0 -> beginning of the file
  - 1 -> current position of the file
  - 2 -> end of file
- By default, the value of `reference_point` is 0
- **TO NOTE**:
  - files opened in text mode only supports `reference_point` value of 0.

- Text mode decodes bytes into characters — an arbitrary byte offset could land in the middle of a multi-byte character (e.g. in UTF-8), corrupting the read.

Some more file methods

`fileno()`

- The Python function `fileno()` method returns the file descriptor (or file handle) of an open file.
- A file descriptor is an unsigned integer that is used by the operating system to identify the open files in an os kernel.
- The file descriptor acts just like the ID that is used to acquire a record stored in a database. Therefore, Python uses it to perform various operations on the files.
- Syntax -> `file_object.fileno()`
  - **params**: the method doesn't require any parameters
  - **return**: the method returns the integer file descriptor
  - usage example

```
f = open('lyrics.txt', 'r')
fid = f.fileno()
print('the file descriptor of', 'lyrics.txt = ', fid)
f.close()
```

  - output: `the file descriptor of lyrics.txt = 3`
  - Every process gets three file descriptors automatically
    1. `0` -> stdin
    2. `1` -> stdout
    3. `2` -> stderr

       The very first file *you* open gets the **next available number**, which is almost always `3`.

`flush()`

- Whenever a write operation is performed on a file, the contents are first written in the internal buffer; which is then transferred into the destination file once the buffer is full.
- This is done to prevent frequent system calls for every write operation
- Once the file is closed, Python automatically flushes the buffer.

- The Python File `flush()` method flushes the internal buffer explicitly, without needing it to be full.
- Syntax -> `file_object.flush()`
  - params: the method doesn't require any parameters
  - return: the method doesn't return any value (or `None`)
- usage example

```
f = open('lyrics.txt', 'a')
f.write('\nheroes who never yield!')
f.flush()
f.close()
```

isatty()

- `isatty()` method checks whether the file stream is interactive or not
- The file stream is said to be interactive if the file is connected to the "tty" type device.
- "tty" is an abbreviation for "Teletypewriter" and refers to an interactive terminal device.
- Syntax -> `file_object.isatty()`
  - params: the method doesn't accept any parameters
  - returns: the method returns a boolean value
- usage example

```
import sys

print(sys.stdin.isatty())    # True
```

```
f = open('/dev/tty')
print(f.isatty())    # True
f.close()
```

truncate()

- `truncate()` method truncates or shortens a file's size
- `truncate(size)` sets the file to exactly `size` bytes. If the file was larger, the extra data is cut off and lost. If the file was

smaller, the new space is filled with null bytes (`\x00`).

- The default value of this argument is the current file position
- Syntax -> `file_object.truncate([size])`
  - **params**: `size` -> optional, The size at which a file is to be truncated
  - **return**: the method doesn't return anything or `None`
- usage example
  `lyrics2.txt`

```
sec for st.edmunds,
winners in every field.
cheer all cheer for st.edmunds,
heroes who never yield!
```

- original file size = 99
  `test.py`

```
f = open('./lyrics2.txt', 'a')
f.truncate(50)
f.close()
```

`lyrics2.txt` after...

```
sec for st.edmunds,
winners in every field.
cheer
```

Directory methods

- Directory is a collection of files of the same or different formats
- Python has a variety of built-in functions in the `os` module to manage directories
- Managing directories involve creating, removing and changing directories

`mkdir()` function

- This function is used to create directories in the path provided as the argument

- The minimal syntax is as `os.mkdir(path)`

```
import os

os.mkdir('./CollegeFiles')
print('Directory created successfully')
```

- The above code creates the directory named `CollegeFiles` in the current working directory

`getcwd()` function

- This function is used to display the current working directory
- Syntax -> `os.getcwd()`
- usage example

```
import os

print(os.getcwd())    # /home/abanggi/python/files
```

`chdir()` function

- The `chdir()` function is used to change the current directory
- Syntax -> `os.chdir(path)`
- A `WindowError` is generated if the path specified doesn't exist
- usage example

```
import os

print('current working directory', os.getcwd())
# output → /home/abanggi/python/files
os.chdir('/home/sirah/Documents')
print('current working directory', os.getcwd())
# output → /home/sirah/Documents
```

`rmdir()` function

- The `rmdir()` function is used to remove or delete a directory
- Syntax -> `rmdir(path)`

- This function can only delete directories which are empty
- usage example

```
import os

os.rmdir('./TestDirectory')
print('Directory removed successfully!')
```

- If the directory is not empty it generates an `OSError` exception